

Methodological Protocol

A Computational Protocol for Statement Classification within the Institutional Grammar Framework

Version 1.0, October 2026

Authors

Karina Arias-Yurisch

Escuela de Gobierno, Pontificia Universidad Católica de Chile

Daniel Toro-Cabrera

Facultad de Ciencias Físicas y Matemáticas, Universidad de Chile

Suggested citation

Arias-Yurisch, K., & Toro-Cabrera, D. (2026). *A Computational Protocol for Statement Classification within the Institutional Grammar Framework* (Version 1.0).

https://github.com/dlatorocabrera/IG_protocol_project

Funding

This work was supported by the Concurso de Investigación y Creación Avanza UC 2025 (grant AV25365), Dirección de Investigación, Vicerrectoría de Investigación y Postgrado, Pontificia Universidad Católica de Chile.

License

© 2026 the authors. This work is licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

<https://creativecommons.org/licenses/by/4.0/>

INTRODUCTION

This protocol proposes a computational workflow for automatically identifying and classifying candidate statements in legal and policy documents as regulative, constitutive, or non-institutional, following the definitions of the Institutional Grammar framework. The objective of this protocol is to classify candidate statements; its scope extends from text preparation to that classification stage. The workflow combines rule-based procedures, supervised machine-learning models, and human validation to support a transparent, reproducible, and scalable process for statement classification within the Institutional Grammar framework.

Institutional Grammar framework

The Institutional Grammar (IG) was introduced by Crawford & Ostrom (1995) as a theoretical syntax for systematically representing the linguistic content of institutions. From this perspective, institutions encompass enduring patterns of human action structured by rules, norms, shared strategies, and the material conditions in which social interactions occur. Institutional statements express shared constraints or opportunities that prescribe, permit, or advise actions or outcomes for individual or collective actors. By decomposing these statements into standardized syntactic components, the IG provides a common unit of analysis for describing and comparing institutional arrangements across documents, policy domains, jurisdictions, and periods.

Subsequent developments of the framework distinguish between regulative and constitutive institutional statements. Regulative statements prescribe or constrain the actions of actors, whereas constitutive statements create, define, modify, or assign properties, roles, relationships, or authority to entities within an institutional system (Frantz & Siddiki, 2021). In regulative statements, the ABDICO syntax identifies the Attribute (A), or actor to whom the statement applies; the Deontic (D), which indicates whether an action is required, permitted, or forbidden; the Aim (I), or action prescribed by the statement; the Object (B), which receives or is affected by that action; the Context (C), which identifies the circumstances under which the statement applies or the action is performed; and the Or Else (O), which specifies a consequential statement associated with compliance or noncompliance. IG 2.0 further differentiates Context into Activation Conditions, which determine when the statement as a whole becomes applicable, and Execution Constraints, which qualify how, where, when, or to what extent the Aim is performed. When an Activation Condition or Execution Constraint is expressed as a nested institutional statement, it may contain its own action and Object. Constitutive statements use a corresponding syntax oriented toward the entities, properties, and functions that configure the institutional system, and institutional texts may contain nested or hybrid combinations of regulative and constitutive statements (Frantz & Siddiki, 2021). The present protocol introduces these distinctions and the ABDICO architecture as the conceptual framework for statement classification.

The IG has been applied to the analysis of laws, regulations, policies, and other formal and informal institutional arrangements. A review by Dunlop et al. (2019) found that early applications primarily used the grammar to examine the textual configuration of policy documents and institutional rule systems, particularly in environmental and common-pool-resource governance. The authors nevertheless caution that component counts and descriptive coding provide limited explanatory value unless they are connected to theoretically grounded research questions. More recent assessments emphasize the methodological versatility of the IG and its ability to support qualitative, quantitative, comparative, and computational analyses of institutional design (Siddiki & Frantz, 2022). Reviewing 48 empirical studies, Pieper et al. (2023) show that IG-coded data have been used to operationalize complex institutional concepts such as polycentricity, discretion, authority, and compliance. These applications demonstrate the value of the IG as a transparent and systematic approach for converting text-based institutional information into structured data, while also highlighting the importance of theoretical interpretation and human validation.

Automation of Institutional Grammar Coding

The increasing application of Institutional Grammar coding has highlighted an important practical limitation: manually identifying institutional statements and decomposing them into their syntactic components requires considerable time, specialized training, and repeated human judgment. Computational methods can reduce some of these costs and support the analysis of larger document collections. However, the existing literature does not provide a single method that automates the entire coding process. Instead, different studies address particular operations within a broader workflow, using combinations of rule-based procedures, natural language processing, supervised machine learning, dictionaries, and human validation. The literature therefore addresses different stages of Institutional Grammar automation rather than one complete end-to-end solution.

The following table summarizes the principal strategies identified in the literature:

Automation task	Approach	Implementation and purpose	References
1. Text preparation and corpus structuring	Rule-based text cleaning	Customized search functions and regular expressions remove formatting noise, irregular punctuation, headings, and other structural artifacts before linguistic processing.	Rice et al. (2021)
	Document-structure filtering	Document-specific markers delimit the substantive regulatory text. Titles, recitals, tables, and appendices can be excluded when they fall outside the analytical scope.	Brandsma et al. (2025)
	NLP-assisted cleaning and entity harmonization	Text-processing and named-entity-recognition tools support character cleaning, acronym expansion, actor identification, and harmonization of different names referring to the same organization.	Galik & Johnson (2026)
2. Candidate-statement segmentation	NLP sentence segmentation	Stanford CoreNLP separates documents into sentences and adds part-of-speech and dependency annotations.	Rice et al. (2021)
	Sentence and token segmentation	spaCy separates text into sentences and tokens before producing part-of-speech tags and dependency trees.	Vannoni (2022)
	Manual statement delimitation	Statements are manually delimited before classification and component annotation. This provides greater interpretive control but limits scalability.	Wróblewska et al. (2023)
3. Identification and classification of relevant statements	Supervised machine-learning classification	TF-IDF representations, selected word n-grams, and a Random Forest classifier distinguish regulative from constitutive statements.	Wróblewska et al. (2023)
	Rule-based dependency parsing	Linguistic rules applied to dependency structures identify statements that satisfy predefined regulatory characteristics.	Brandsma et al. (2025)

Automation task	Approach	Implementation and purpose	References
	Transformer-based classification	BERT, LEGAL-BERT, and InLegalBERT are trained and evaluated as alternative classifiers for identifying regulatory statements.	Brandsma et al. (2025)
	Keyword-based selection	Keywords identify statements associated with a particular substantive policy topic. This supports topical selection but does not, by itself, distinguish institutional-statement types.	Kim & Collins (2026)
Future extension: extraction of IG components (outside the current protocol scope)	Dictionary-based extraction	Manually refined dictionaries identify actors, rule categories, and deontic operators. Identified components can subsequently be represented as dyadic relations or networks.	Heikkila & Weible (2018)
	Rule-based linguistic extraction	Part-of-speech tags and dependency relations generated with spaCy, Stanza, or cleanNLP are mapped to Institutional Grammar components through explicit linguistic rules.	Kim & Collins (2026); Vannoni (2022); Wróblewska et al. (2023)
	Entity, verb, and deontic extraction	Named-entity recognition, actor disambiguation, selective verb extraction, and deontic dictionaries identify selected elements of institutional arrangements.	Galik & Johnson (2026)
	Supervised token classification	Word-level linguistic and contextual features are classified into IG component categories using neural-network models.	Rice et al. (2021)
	Semantic Role and Labeling and coreference resolution	Semantic roles such as agents, actions, and recipients are mapped to corresponding IG components. Coreference resolution recovers actors or objects referenced across sentences.	Bushouse et al. (2026)

General Pipeline of the Protocol

The protocol is organized into three sequential and interconnected tasks. Task 1, Text Preparation and Corpus Structuring, registers the source documents, extracts and normalizes their text, evaluates extraction quality, and represents their legal structure in a machine-readable hierarchy. Task 2, Candidate Statement Segmentation, divides substantive text-bearing elements into candidate statements while preserving their location, structural context, and links to document elements. Task 3, Institutional Statement Classification, uses a human-validated labeled corpus and supervised machine-learning methods to classify each candidate as a regulative statement, a constitutive statement, or a non-institutional statement. Each task produces structured and traceable outputs that serve as inputs for the following task while retaining links to the original document and intermediate processing events. Together, these tasks establish an auditable pipeline from source documents to classified candidate statements. The classified statements provide a basis for future development of a broader Institutional Grammar coding process, including ABDICO component extraction.

This document should be understood as an iterative, human-in-the-loop workflow proposal. Automated outputs are treated as provisional processing and coding proposals rather than final authoritative interpretations. Human review is incorporated to assess extraction quality, validate ambiguous classifications, refine parsing and coding rules, improve computational models, and progressively expand a validated dataset for future training and evaluation. This iterative process allows the workflow to improve as additional documents and manually reviewed cases become available while maintaining transparency, traceability, and researcher oversight.

TASK 1: TEXT PREPARATION AND CORPUS STRUCTURING

The purpose of this task is to transform the original legal or policy documents into a clean, structured, and traceable corpus that can be used in later stages of segmentation and statement classification.

Sub-task 1.1: Document Intake and Corpus Inventory

The first step is to create a structured inventory of all documents included in the corpus, establishing the provenance, legal jurisdiction, format, and basic characteristics of each source document before any text extraction or automated coding is performed.

Each document must receive a unique identifier that remains unchanged throughout the workflow, linking the original file to its extracted text, legal structure, candidate statements, statement-classification outputs, validation records, and manual corrections. To support future auditing, all source files, processing events, generated outputs, corrections, and parser or model versions should be preserved through stable identifiers, versioned paths, timestamps, and integrity hashes, so that a reviewer can trace which input, script version, parameters, and intermediate artifact produced each downstream record without overwriting earlier runs.

The complete document-inventory field dictionary and its SQLite and JSON representations are documented in the supporting document *Technical Database and Data-Structure Specification*.

The resulting document inventory will function as the corpus-level metadata table. It should be created before text extraction and updated only when new documents are incorporated, excluded, or corrected. This table provides the basis for traceability and reproducibility throughout all subsequent stages of the automated Institutional Grammar workflow.

Sub-task 1.2: Text Extraction

This sub-task extracts raw text from the source PDF and stores it in page-level format while preserving the original document and creating a separate raw-text version linked through `document_id`. The output is not yet interpreted, normalized, or structurally classified. For searchable PDFs, extraction can be implemented with `pdfplumber`. The result should preserve page boundaries and create both a raw text file and a page-level JSON file. At this stage, the workflow should not substantially alter or correct the text; its objective is to obtain the most complete textual representation possible while retaining the source format and extraction procedure.

The extraction method should be selected according to the original file format recorded in the document inventory. The following table provides examples of extraction methods¹.

Original file format	Extraction method	Protocol instruction
Searchable PDF	Direct PDF-to-text extraction using <code>pdfplumber</code> .	Use <code>pdfplumber</code> to extract text page by page. Preserve page order and, where available, layout or positional information.
Scanned PDF	Optical Character Recognition (OCR).	Recommended when the PDF contains images rather than an accessible text layer. The OCR language configuration should match the corpus language.

¹ The implemented extraction route covers searchable PDFs using `pdfplumber`. OCR and mixed-document routes are retained as methodological options but remain planned extensions in this version.

Original file format	Extraction method	Protocol instruction
Mixed PDF	Direct PDF extraction followed by OCR only for pages with missing or unusable text.	First apply pdfplumber. Identify pages with empty, incomplete, or corrupted extraction and apply OCR only to those pages.

The implementation should prioritize the least invasive method that produces usable text. Direct extraction should be preferred when a reliable text layer already exists. OCR should be used only when direct extraction is unavailable, incomplete, or of insufficient quality.

Extraction workflow

1. Identify the original file format.
2. Select the extraction route.
3. Extract text at the smallest available structural level preserving the original source.
 - For searchable PDF documents, text can be extracted using pdfplumber, as in Turaga et al. (2025). The extraction should produce a raw text file that preserves page boundaries through explicit page markers, for example <PAGE number="17">. This page-level text file constitutes the direct output of the extraction process.
 - For computational processing and traceability, the extracted text should also be stored in a JSON file containing the document identifier, extraction metadata, and the raw text associated with each page. JSON is recommended in this protocol because it can preserve the relationship between the extracted text and relevant metadata, including page number, document identifier, extraction method, and extraction date. This is consistent with the use of structured JSON input in DODFMiner (Guimarães et al., 2024).
4. Each execution is registered as an extraction event linked to document_id. The event records the method, tool and version, page counts, output paths, integrity information, status, and notes; its complete schema is provided in the technical specification.

Sub-task 1.3: Extraction-Quality Assessment

Because OCR errors, heterogeneous document formats, footnotes, cross-references, and tables may introduce noise into subsequent classification and extraction stages, this sub-task evaluates whether the extraction process has produced a technically reliable representation of the original document.

Problems such as repeated headers, spacing inconsistencies, line-break hyphenation, and other formatting artifacts will be addressed during text normalization. However, pages from which no text was extracted require separate identification because they may indicate an extraction failure, an image-based page, or the need to apply OCR.

The assessment can be run in Python using the page-level JSON output from Sub-task 1.2. The procedure should compare the original and extracted page counts, calculate the amount of text extracted from each page, and flag blank or near-blank pages. It produces a quality-checked JSON file that retains the original page-level text and adds page-specific quality flags and a document-level extraction summary.

The following table presents the automated technical checks applied in this process.

Quality check	Description	Output
Page-count comparison	Compares the number of pages in the original document with the number of page records in the extracted JSON file.	Flag when the two counts do not match.

Quality check	Description	Output
Blank-page detection	Identifies pages for which <code>raw_text</code> is empty, null, or contains only blank spaces.	Flag as <code>blank_extraction</code> .
Near-blank-page detection	Identifies pages with very limited extracted text, based on a predefined minimum number of non-space characters.	Flag as <code>near_blank_extraction</code> .

Pages flagged for manual review should include both a machine-readable reason code and a free-text note explaining the reason for review. This allows later reviewers to distinguish between blank extraction, near-blank extraction, encoding problems, page-count mismatches, and complex non-textual content such as tables, figures, signatures, or scanned material.

A blank or near-blank extraction should not automatically be interpreted as an empty source page. The original document may contain a scanned page, image-based table, figure, signature, or other complex element from which standard text extraction cannot recover text. These pages should therefore be flagged for later inspection, OCR, or preservation as non-textual content.

Sub-task 1.4: Text Normalization

The purpose of text normalization is to reduce technical noise introduced by PDF extraction, optical character recognition (OCR), heterogeneous document formats, and source repositories while preserving the original extracted text. This sub-task receives the quality-checked page-level JSON produced in Sub-task 1.3, retains the original `raw_text` field without modification, and applies all normalization transformations to a separate `normalized_text` field. The resulting cleaned text can then be used for legal-structure identification and candidate statement segmentation while maintaining traceability to the raw extraction.

1.4.1. Character and Encoding Normalization

The first step should standardize the character representation of the text. This includes applying Unicode normalization, correcting irregular encoding artifacts, standardizing quotation marks and dashes, and removing non-linguistic special characters when they are clearly technical noise. This can be implemented in Python using `unicodedata` and regular expressions, keeping the procedure transparent, reproducible, and easy to document (Galik & Johnson, 2026; Saketos et al., 2024).

For Spanish legal texts, accents should be preserved in the main normalized text. Accent removal should not be applied by default because diacritics may be relevant for names, legal terms, and readability. If accent-insensitive matching is needed later, the protocol may create an auxiliary field, but the main `normalized_text` should preserve accents.

The following table illustrates the types of character and encoding normalization that can be applied during this step.

Normalization process	Example extracted text	in	Normalized output	Purpose
Unicode composition normalization (NFC)	resolucio\u0301n		resolución	Converts decomposed characters into a standard composed form. Recommended for the main <code>normalized_text</code> .
Unicode compatibility normalization	fiscalización		fiscalización	Converts compatibility characters, such as ligatures, into standard characters. Useful when PDF extraction produces typographic variants.

Normalization process	Example extracted text	in	Normalized output	Purpose
Quotation-mark standardization	"El Ministerio" / 'autoridad'		"El Ministerio" / 'autoridad'	Standardizes typographic quotation marks to plain quotation marks for easier matching.
Dash standardization	Ambiente —en adelante—		Ambiente -en adelante-	Converts long dashes or en-dashes into a consistent dash format.
Space normalization	Artículo 17		Artículo 17	Converts non-breaking spaces or irregular spaces into ordinary spaces.
Soft-hyphen removal	descontaminación		descontaminación	Removes invisible hyphenation marks introduced by PDF extraction.
Replacement-character detection	regulaci□n		flagged for review	The character □ usually indicates an encoding problem. It should be flagged rather than automatically corrected.
Zero-width character removal	Ministerio		Ministerio	Removes invisible characters that may interfere with tokenization or matching.
Accent-insensitive auxiliary version	Secretaría deberá informar la resolución		secretaria debera informar la resolucion	Should be stored only in an auxiliary field, such as <code>matching_text</code> , not in the main <code>normalized_text</code> .

1.4.2. Spacing, Line Break, and Hyphenation Cleaning

The second step should correct formatting artifacts generated during text extraction. This includes removing excessive spaces, standardizing line breaks, reducing repeated blank lines, and joining words incorrectly split by line-break hyphenation.

This operation should be applied carefully. Not every line break should be removed, because some line breaks may indicate legal structure, such as articles, paragraphs, numbered lists, lettered items, or headings. Therefore, structural markers such as *Artículo*, *CAPÍTULO*, section headings, numbered items, and lettered items should be preserved whenever they appear in the text.

1.4.3. Removal of Technical Non-Substantive Content

The third step should remove technical or repository-generated content that is not part of the legal document itself. This may include HTML tags, web-navigation elements, repeated repository labels, automatically generated page numbers, or duplicated headers and footers.

However, the implementation should not automatically remove legally meaningful content. Titles, chapters, sections, article headings, definitions, recitals, annexes, footnotes, and tables may provide context for later legal-structure identification and Institutional Grammar coding.

Therefore, the recommended approach is selective removal:

Content type	Recommended treatment
HTML/XML tags	Remove from <code>normalized_text</code> .

Content type	Recommended treatment
Repository headers and footers	Remove when they are repeated technical artifacts.
Page numbers generated by the repository	Remove or store separately.
Legal document title	Preserve.
Chapter, section, and article headings	Preserve.
Definitions	Preserve for later classification.
Tables, figures, annexes, and footnotes	Preserve or flag for structural representation.

1.4.4. Output of the Normalization Step

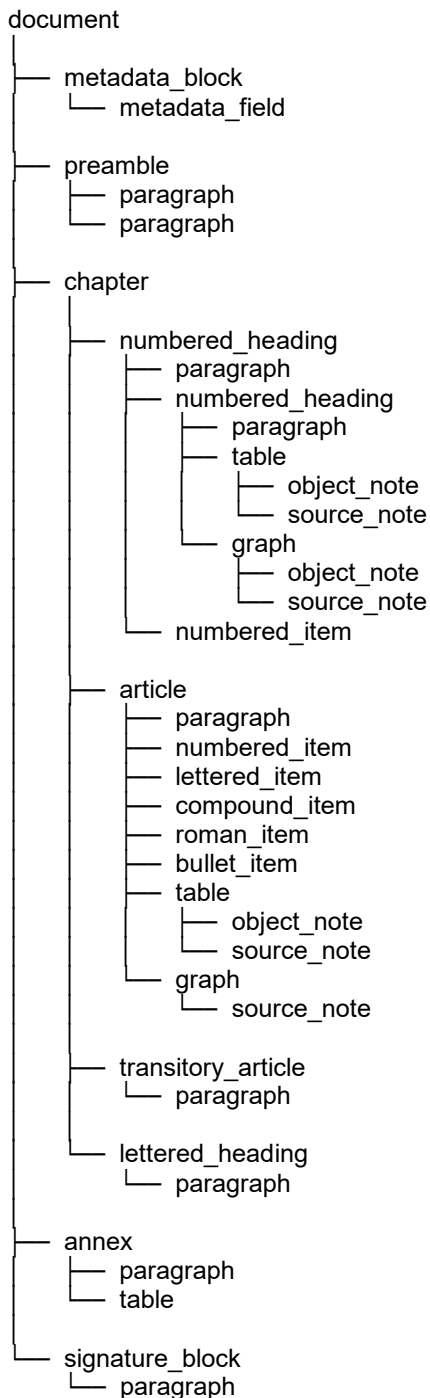
The output of this sub-task should be a normalized page-level JSON file. The file should retain the original `raw_text` from Sub-task 1.2 and the quality flags from Sub-task 1.3, while adding a new `normalized_text` field. The document-level metadata should also record the normalization rules applied, the software used, the date of processing, and any warnings.

Sub-task 1.5: Legal-Structure Identification and Hierarchical Representation

The purpose of this sub-task is to generate a hierarchical representation of the legal document using the normalized page-level corpus produced in Sub-task 1.4, preserving the formal organization of the legal text in a machine-readable structure.

The current implementation uses a rule-based parser to process the normalized page-level JSON created in Sub-task 1.4. The primary text field for structure identification is `normalization.normalized_text`. The `raw_text` field remains preserved for traceability but is not used as the main source for parsing. Each structural element preserves enough information to trace it to the source document and reconstruct the legal hierarchy.

The parser does not infer hierarchy from semantic meaning. It uses the most recent explicit structural context. For example, a paragraph following *Artículo 57* is nested under that article, while an article following *VI.6.1* is nested under that numbered heading. The following scheme shows the hierarchical organization of the document in the JSON file:



The parser follows the next sequence of processes:

1. **Page-level flattening:** Reads each normalized page and converts it into ordered candidate lines. Page numbers are retained so every structural element can preserve `page_start` and `page_end`.
2. **Embedded marker splitting:** Some legal markers may appear inside a line rather than at the beginning because of PDF extraction or normalization. The parser can split embedded preamble markers, such as `Visto:`, `Considerando:`, or `Decreto:`, so they become separate structural anchors.
3. **Metadata block detection:** LeyChile-style metadata fields are detected at the beginning of the document and nested under a `metadata_block`. Examples include `Tipo Norma`, `Fecha Publicación`, `Organismo`, `Título`, `Id Norma`, and `URL`.

4. **Preamble detection:** Preamble labels such as Visto, Considerando, Decreto, and Resuelvo are identified as preamble elements. Their text is stored as paragraph children.
5. **Major legal hierarchy detection:** Chapters, sections, subsections, paragraphs-as-headings, roman-decimal numbered headings, articles, and transitory articles are identified using explicit markers.
6. **Leaf-level content detection:** Paragraphs, numbered items, lettered items, roman items, compound items, bullet items, visual objects, source notes, object notes, and signatures are stored as leaf-level or lower-level elements.
7. **Parent assignment:** Each element is nested under the current legal context. The parser prioritizes article, active numbered heading, subsection, section, chapter, and preamble states when assigning parents.
8. **Output generation:** After all pages are parsed, open element ranges are finalized, page ranges are updated, and the nested and flat outputs are written.

The parser recognizes structural elements only when they are explicitly marked in the normalized text. The following table summarizes the main element types and the rules used to identify them in the current stable version.

Element type	Markers / rule	Decision notes
metadata_block metadata_field	Tipo Norma:, Fecha Publicación:, Fecha Promulgación:, Organismo:, Título:, Tipo Versión:, Inicio Vigencia:, Id Norma:, URL:	Used primarily for LeyChile-style metadata. Metadata fields are nested under one metadata_block.
preamble	Visto:, Considerando:, Decreto:, Resuelvo:	Detected as explicit preamble labels. Text following these labels is stored as child paragraphs.
chapter	CAPÍTULO + roman or numeric label	Label is preserved as CAPÍTULO I, CAPÍTULO II, etc. Title is separated when present.
title / section / subsection	TÍTULO, SECCIÓN, SUBSECCIÓN, PÁRRAFO + roman or numeric label	Used when explicitly present. Titles are cleaned only to remove leading separators.
numbered_heading	Roman-decimal heading such as I.1, I.2.1, III.3, VI.6, VI.6.1	A numbered-heading stack is used to represent nested heading depth.
article	Artículo + number, followed by punctuation/separator or line end	This avoids treating prose references such as “artículo 43 del presente decreto” as new article headings.
transitory_article	Artículo Transitorio, Artículo único Transitorio, or ordinal transitory article	Stored separately from ordinary numbered articles.
numbered_item	1. item text or 1) item text	Four-digit years such as 2016. are excluded to avoid false list items.
compound_item	a.1), a.2), b.1), etc.	Used for nested alphabetic-decimal list markers.
lettered_item	a) item text or a. item text	Only explicit punctuation forms are used in v0.6. Bare letters without punctuation are intentionally left for future versions.
roman_item	i), i., ii), ii., iii), iii., etc.	Used for lowercase roman list items.

Element type	Markers / rule	Decision notes
bullet_item	- item text, – item text, — item text	Used for dash-marked list items.
table / graph / figure / annex	Tabla, Cuadro, Gráfico, Figura, or Anexo followed by a compact label and then : or .	The punctuation requirement prevents prose references to tables or graphs from being classified as new visual objects.
source_note	Fuente:	Nested under the most recent table, graph, or figure when available.
object_note	PI=, CI=, (*), (**), Nota:, Observación	Nested under the most recent visual object when available.
lettered_heading	A. Heading text, B. Heading text	Used for uppercase internal subheadings, especially in later chapters.
signature_block	Anótese, Tómese razón, Publíquese, Comuníquese, Regístrese	Used for signature and promulgation closing material.
paragraph	Default textual unit under the current parent	Used when the text does not match another explicit structural marker.
other	Fallback when no parent or clear structure is available	Assigned uncertain confidence.

Parent assignment is based on the current legal context. The parser uses a state hierarchy rather than semantic inference. When a new structural element is detected, it is attached to the most specific active parent available.

Rule	Parenting decision
Root-level elements	metadata_block, preamble blocks, chapters, and some signature blocks may appear at the root level.
Chapters	A chapter becomes the active high-level parent. New sections, numbered headings, and articles are attached under it until a new chapter appears.
Numbered headings	Roman-decimal headings are nested by depth. For example, I.2.1 is nested under I.2; VI.6.1 is nested under VI.6.
Articles under headings	If a numbered heading is active, a new article is attached under that heading rather than directly under the chapter.
Articles without active numbered heading	The article is attached under the current subsection, section, or chapter.
Paragraphs and items	Paragraphs and list items are attached under the current article when an article is active; otherwise they are attached under the active heading or chapter.
Visual objects	Tables, graphs, and figures are attached under the current parent. Their notes and sources are nested under the visual object.
Annexes	Annexes are attached under the current chapter when this relationship is clear.
Signature block	Closing signature formulas start a signature_block and subsequent signature lines may be appended to it.

Labels and titles are stored separately. For structural headings and visual objects, the parser preserves the visible label and stores the title or caption separately when available. Leading

separators such as :, -, or . are removed from the title, but the wording itself is not rewritten. Capitalized Spanish determiners, contractions, prepositions, conjunctions, connectors, and reflexive or legal sentence openings may act as conservative title-boundary markers; the exact implemented list is documented in the technical specification.

The rule is case-sensitive. Lowercase function words such as: para, con, de, en, or si are not used as title boundaries because they frequently occur inside valid legal headings and captions.

The current parser only identifies table, graph, figure, and annex captions when the label is compact and is followed by a colon or a period. This prevents ordinary prose references from becoming false visual objects. Fuente: lines are stored as source_note elements. When a recent table, graph, or figure has been detected, the source_note is nested under that visual object. If a Fuente line accidentally absorbs the next body sentence, the same capitalized Spanish function-word boundary rule is used to split the source text from the following paragraph.² When a paragraph or list item is split only because of a page break, the next text element is appended to the previous one if it starts with a lowercase letter and appears on a later page.

Sub-task 1.5 produces two complementary database tables. The first table, structure_events, records each execution of the legal-structure parser. It links the structured output to the original document, extraction event, quality-assessment event, and normalization event. It also records the parser tool, method, version, input text field, output file paths, element counts, warning counts, and execution status. This table supports reproducibility by making it possible to identify which version of the parser generated each legal-structure representation.

The structure_elements table stores a flattened representation of the hierarchical JSON generated in Sub-task 1.5. Each row corresponds to one structural element, such as a chapter, article, paragraph, numbered item, table, graph, source note, or signature block. The variables element_id and parent_id preserve the parent-child relationship, while element_order preserves the original document order and nesting_level records the depth of the element within the hierarchy.

Together, structure_events and structure_elements preserve traceability from the normalized text input to the structured JSON and CSV outputs. Their implemented SQL fields, foreign-key relationships, categorical values, and examples are documented in the companion technical specification.

² Notes such as Pl=, Cl=, (*), (**), Nota:, and Observación are stored as object_note elements when they occur in the context of a recent visual object.

TASK 2: SEGMENTATION INTO CANDIDATE STATEMENTS

The purpose of this task is to transform the structured legal corpus produced in Task 1 into sentence-level units that can be used in later stages of statement classification. Its purpose is limited to producing traceable candidate textual units.

A candidate statement is defined here as a sentence-level textual unit extracted from a substantive structural element of the legal document. These candidate statements are not yet confirmed institutional statements. They are provisional units that may later be classified as regulative, constitutive, or non-institutional in the context of Institutional Grammar coding.

The task receives as input the completed legal-structure representation generated in Sub-task 1.5, principally `structure_elements` and its corresponding hierarchical JSON rather than the original PDF or raw page text. Segmentation is applied independently to substantive text-bearing elements—paragraphs, numbered items, lettered items, compound items, roman items, and bullet items—while metadata, headings, tables, graphs, figures, source notes, object notes, and signature material remain preserved as contextual structure and are not segmented as statements. This keeps every candidate linked to its `element_id`, `parent_id`, element type, order, page range, review flags, and source metadata.

Sub-task 2.1: Segmentation Method

The current implementation uses a Spanish sentence-segmentation procedure based on spaCy. The default model is `es_core_news_md`. This model is used because the corpus is in Spanish and sentence boundaries depend on Spanish punctuation, abbreviations, and syntactic conventions. spaCy produces initial sentence boundaries, which are then reviewed through deterministic legal-text rules described in the following sub-task. Post-processing is necessary because general sentence segmentation may incorrectly split Spanish legal references, abbreviations, numbered markers, decree references, or text following semicolons and colons.

Sub-task 2.2: Legal-Boundary Post-Processing

After spaCy generates preliminary sentence boundaries, the implementation applies conservative post-processing rules to prevent false splits. The purpose of these rules is to protect boundaries that are technically unsafe for Spanish legal documents.

The following boundary cases are protected:

Boundary case	Decision
Semicolon or colon	Do not treat the boundary as a final sentence split when the following text continues the same legal unit.
Non-terminal line break	Merge when the left side does not end in final sentence punctuation.
Ellipsis continuation	Merge when an ellipsis is followed by lowercase continuation.
Article abbreviations	Protect abbreviations such as Art. and Arts. when followed by a legal reference.
Legal decree abbreviations	Protect expressions such as D.S., D.L., and D.F.L..
Number abbreviations	Protect expressions such as N°, N.º, Núm., and No..
Personal titles	Protect expressions such as Sr. and Sra..
etc. abbreviation	Avoid splitting when etc. is followed by continuation text.
Initials and multi-period abbreviations	Avoid treating initials or multi-period abbreviations as sentence endings.

Boundary case	Decision
Numbered legal markers	Avoid treating isolated markers such as 1. or 2) as independent sentences.
Numeric expressions	Avoid splitting inside decimal numbers, dotted numerical expressions, or similar patterns.

These rules are intentionally conservative. They only merge boundaries when there is a clear technical reason to believe that the split is unsafe.

Sub-task 2.3: Candidate Statement Creation

For each eligible source element, the segmentation procedure creates one or more candidate statements. Each candidate statement receives a stable identifier and preserves its relationship to the original legal structure.

The candidate statement keeps:

Field	Purpose
statement_id	Stable identifier assigned to each candidate statement.
segmentation_id	Identifier of the segmentation run that produced the statement.
document_id	Source document identifier.
structure_id	Legal-structure event used as input.
source_element_id	Structural element from which the statement was extracted.
source_parent_id	Parent element of the source element.
source_element_type	Type of source element, such as paragraph or numbered item.
source_element_order	Original order of the source element in the legal structure.
sentence_order_within_element	Position of the sentence inside the source element.
statement_order	Global order of the candidate statement in the document.
statement_text	Text of the candidate statement.
source_char_start / source_char_end	Character offsets in the original element_text.
page_start / page_end	Page range inherited from the source structural element.
page_assignment_method	Method used to assign page location.
segmentation_method	Method used to create the candidate statement.
segmentation_status	Status of the individual candidate statement.
requires_manual_review	Indicates whether the candidate should be checked manually.
review_reason_code	Machine-readable reason for review.
review_reason_note	Free-text explanation of the review flag.

Sub-task 2.4: Review Flags and Segmentation Warnings

The segmentation procedure does not automatically discard uncertain candidate statements. Instead, it flags them for manual review. This is consistent with the broader human-in-the-loop

logic of the protocol, in which automated outputs are treated as initial coding proposals that can be reviewed and refined.

A candidate statement may be flagged for review when:

Review reason code	Meaning
very_short_sentence	The candidate has fewer than 20 characters or fewer than three word-like tokens.
sentence_starts_lowercase	The candidate begins with a lowercase letter, which may indicate an over-split continuation.
possible_abbreviation_split	The accepted boundary follows a known abbreviation and may need review.
missing_terminal_punctuation	The candidate lacks final sentence punctuation. This may be acceptable for list items but should be checked.
source_element_review:<code>	The candidate inherits a manual-review flag from the source structural element.

Sub-task 2.5: Output Files and Database Records

Task 2 produces three main outputs.

1. JSON file containing document metadata, structure-event metadata, segmentation metadata, quality checks, warning summaries, protected boundary merges, and the full list of candidate statements.
2. CSV file with one row per candidate statement. This file is designed for inspection, manual review, and simple comparison across segmentation runs.
3. Results in SQLite using two tables: Task 2 registers one `segmentation_events` record per run and one `candidate_statements` record per accepted sentence. The event preserves the structure input, spaCy model and versions, counts, output paths, status, and notes; candidate rows preserve source-element lineage, ordering, page range, character offsets, segmentation method, and review flags.

TASK 3: CLASSIFICATION OF CANDIDATE STATEMENTS

The purpose of this task is to classify the candidate statements produced in Task 2 according to their institutional relevance and type. Each candidate statement is assigned to one of three mutually exclusive categories: regulative, constitutive, or non-institutional.

Regulative statements prescribe, permit, prohibit, authorize, or otherwise regulate an actor's actions or decisions. Constitutive statements define or establish institutional entities, roles, properties, functions, or arrangements. Non-institutional statements contain descriptive, contextual, evidentiary, fragmentary, or other content that does not constitute an ABDICO institutional statement.

The task receives as its principal input the `candidate_statements` table produced in Task 2 and a human-validated labeled corpus containing examples of the three classification categories. Every candidate retains its original statement text, stable identifier, source structural element, document identifier, page location, and legal context. Classification therefore adds a new analytical layer without modifying the candidate statement or breaking its connection to the source document.

The current implementation uses supervised machine learning with term frequency–inverse document frequency (TF-IDF) representation. This approach is informed by Wróblewska et al. (2023), who use textual features and supervised classification to distinguish between regulative and constitutive institutional statements. The present protocol extends the classification problem by adding a non-institutional category, allowing the model to distinguish codable institutional statements from other textual units generated during sentence segmentation.

Sub-task 3.1: Input Dataset

Task 3 receives two separate but structurally compatible input datasets. The first is a **human-validated labeled corpus**, used for model training and evaluation. The second is an **unlabeled classification corpus**, containing the candidate statements generated in Task 2 to which the trained model will be applied.

Human-Validated Labeled Corpus

The labeled corpus provides the reference observations required to develop and evaluate the supervised classification model. Each observation contains an original candidate statement and a human-validated classification of regulative, constitutive, or non_institutional. This corpus is subsequently divided into training and test partitions as described in Sub-task 3.2.

For the current implementation, the labeled corpus was constructed by aligning the candidate statements generated in Task 2 with an existing manually coded dataset and manually reviewing the resulting correspondences. Candidate statements that could not be matched were not automatically classified as non-institutional. A label was incorporated into the modeling corpus only when it could be supported by the existing coded dataset or by explicit human validation. The detailed matching, alignment, and review procedure is documented separately in the GitHub project³.

At minimum, each observation in the labeled corpus should contain the following information:

Field	Purpose
statement_id	Stable identifier of the candidate statement generated in Task 2.
statement_text	Original candidate-statement text used as the basis for classification.
class_label	Human-validated classification: regulative, constitutive, or non_institutional.

³ https://github.com/dlatorocabrera/IG_protocol_project.git

Field	Purpose
document_id	Identifier of the source document.
segmentation_id	Identifier of the Task 2 segmentation event that generated the candidate statement.
source_element_id	Identifier of the structural element from which the statement was extracted.
source_element_type	Type of source element, such as paragraph, numbered_item, or lettered_item.
duplicate_group_id	Identifier used to keep exact or normalized-text duplicates together during dataset partitioning.
label_source	Provenance of the validated classification, such as an existing manually coded dataset or subsequent human review.
dataset_version	Version of the labeled corpus in which the observation is included.

Unlabeled Classification Corpus

The unlabeled classification corpus contains the candidate statements produced in Task 2 for which an institutional classification is required. These observations retain the original statement text and their links to the source document, segmentation event, and legal-structure element, but they do not require a human-validated class_label. After model development and evaluation, the selected model is applied to this corpus to generate predicted classifications and associated confidence information.

At minimum, each observation in the unlabeled classification corpus should contain the following information:

Field	Purpose
statement_id	Stable identifier of the candidate statement generated in Task 2.
statement_text	Original candidate-statement text submitted to the classification model.
document_id	Identifier of the source document.
segmentation_id	Identifier of the Task 2 segmentation event that generated the candidate statement.
source_element_id	Identifier of the structural element from which the statement was extracted.
source_element_type	Type of source element from which the candidate statement was generated.
dataset_version	Version of the candidate-statement corpus submitted for classification.

In both input datasets, the original statement_text should remain unchanged. Normalized or simplified representations may be created for duplicate detection, alignment, feature generation, or model processing, but they should be stored separately and should never replace the original candidate text. Model predictions should likewise be stored separately from class_label, ensuring that human-validated labels and automated classifications remain distinguishable and independently traceable.

Sub-task 3.2: Dataset Partitioning

The labeled corpus should be divided into separate partitions before text vectorization, feature selection, or model training. The purpose of this separation is to ensure that model development

and evaluation are conducted on different observations and to reduce the risk of information leakage. This follows the general supervised-classification approach used by Wróblewska et al. (2023), in which model performance is evaluated using data not employed in fitting the classifier.

The current implementation uses an 80/20 stratified partition:

- 80% of the observations are assigned to the training partition.
- 20% are assigned to a test partition.
- The distribution of regulative, constitutive, and non-institutional statements is preserved as closely as possible across both partitions.
- A fixed random seed should be used so that the partition can be reproduced.

Before assigning observations to the partitions, exact and normalized-text duplicates should be identified. Statements belonging to the same duplicate group must be assigned to the same partition. This prevents a classifier from being trained and evaluated on identical or functionally equivalent observations.

The partitioning procedure should follow this sequence:

1. Confirm that every eligible observation has a stable `statement_id`, an original `statement_text`, and a human-validated class label.
2. Exclude or hold observations with unresolved labels or missing candidate-statement identifiers.
3. Identify exact and normalized-text duplicate groups.
4. Apply a stratified partition based on the three classification labels.
5. Assign every member of a duplicate group to the same partition.
6. Verify the resulting class distributions and confirm that no duplicate group appears in both partitions.
7. Store the partition assignments and the parameters used to generate them.

The test partition should remain excluded from vectorizer fitting, feature selection, cross-validation, and model training. It may be used after internal model development to compare candidate configurations. Because the current test partition contributed to the selection of the prototype model, it should not be interpreted as a final independent test set. A later evaluation should therefore apply the frozen pipeline to a completely new corpus that did not contribute to matching, labeling, model comparison, or prototype selection.

The partitioning process creates two related database records:

1. `split_events`: preserves the dataset version, source-review inputs, eligibility and class information, partition parameters, duplicate-control and quality checks, output paths and hashes, execution status, timestamp, and notes.
2. `split_assignments`: preserves the split identifier, candidate identifier, duplicate-group identifier, human-validated gold label, assigned partition, random seed, and creation timestamp. The complete field dictionaries and accepted values for both tables are provided in the companion technical specification.

Once created and used for model development, the partition assignments should remain unchanged. Any later modification to the labeled dataset should generate a new dataset version and, when appropriate, a new partitioning event.

Sub-task 3.3: TF-IDF Text Representation

After partitioning, candidate statements should be transformed into numerical representations that can be processed by supervised machine-learning models. The current implementation uses Term

Frequency–Inverse Document Frequency (TF-IDF) to represent the lexical content of each candidate statement.

TF-IDF assigns greater weight to terms and expressions that are informative within the corpus while reducing the influence of terms that occur frequently across many statements. This representation is appropriate for the current classification task because regulative, constitutive, and non-institutional statements may differ in their use of verbs, modal expressions, institutional actors, legal concepts, and recurring combinations of words.

The TF-IDF vectorizer must be fitted exclusively on the training partition. The fitted vectorizer is then applied unchanged to the test partition. Consequently, statements in the test partition do not contribute to vocabulary creation, document-frequency calculations, inverse-document-frequency weights, or any other fitted vectorizer parameter.

The current implementation applies the following vectorization decisions:

Vectorization decision	Implementation
Input unit	One candidate statement per observation.
Source text	Original statement_text generated in Task 2.
Analyzer	Word-based analysis.
N-gram range	Unigrams, bigrams, and trigrams.
Case	Text is converted to lowercase for vectorization.
Accents	Preserved.
Stop words	Preserved.
Spanish single-character words	a, e, o, u, and y are retained.
Stemming	Not applied.
Lemmatization	Not applied.
Weighting	Term frequency–inverse document frequency.
Normalization	L2 normalization.
Inverse-document frequency	Smoothed IDF.

The preservation of stop words is intentional. Spanish function words may form part of deontic expressions, institutional constructions, or recurring lexical patterns that contribute to distinguishing between statement classes. Similarly, accents are retained because they form part of the original Spanish orthography and may distinguish otherwise similar words.

Unigrams represent individual words, while bigrams and trigrams represent sequences of two or three consecutive words. The inclusion of these n-grams allows the model to use both isolated lexical terms and short multiword expressions. For example, expressions associated with obligations, permissions, definitions, institutional actors, or legal conditions may be more informative when represented as word combinations rather than as independent tokens.

The vectorization process should follow this sequence:

1. Fitting: load the statement identifiers, original text, validated labels, and partition assignments created in Sub-task 3.2; fit the TF-IDF vocabulary and weights using only training statements.
2. Transformation: transform the training statements and apply the same fitted vectorizer, without refitting, to the test statements.

3. Validation: verify matrix dimensions, nonzero values, row order, statement identifiers, partition membership, and the absence of test-derived vocabulary information.

4. Artifact registration: store the fitted vectorizer, sparse matrices, feature index, row mappings, parameters, validation results, paths, hashes, and processing metadata as versioned outputs.

Each run is represented by one `vectorization_events` row, while its six external files are registered in `vectorization_artifacts`. `split_id` links the vectorization to the partitioning event, and `vectorization_id` links it to the training and test matrices, row-index files, feature index, and fitted vectorizer. Large matrices and fitted binary objects remain outside SQLite; paths, roles, sizes, and SHA-256 hashes preserve traceability. Complete fields and artifact schemas are provided in the companion technical specification.

Sub-task 3.4: Model Implementation, Training, and Comparison

After vectorization, the training feature matrix and its corresponding human-validated labels are used to train a supervised multiclass classifier. The classifier assigns each candidate statement to one of the three categories defined for Task 3: regulative, constitutive, or non-institutional.

The implementation does not require one specific classification algorithm. Researchers may use Random Forest, Support Vector Machine, logistic regression, pretrained transformer models, or other supervised classifiers, provided that the same data-partitioning, information-leakage, evaluation, versioning, and traceability requirements are maintained. This allows the classification method to be adapted as the labeled corpus grows or as more suitable models become available.

Candidate model family	Use in the pipeline	Status
Random Forest	Classical ensemble classifier; current prototype family.	Implemented
Support Vector Machine / logistic regression	Alternative sparse-feature classifiers within the same split and evaluation design.	Permitted alternative
Pretrained transformer	Alternative contextual classifier when data and computational resources support fine-tuning.	Permitted alternative

The current implementation uses Random Forest classifiers. This decision is informed by Wróblewska et al. (2023), who combine TF-IDF features, supervised classification, and feature-selection procedures to classify institutional statements. Random Forest also allows nonlinear relationships between textual features, can be trained with or without class weights, and provides measures of feature importance.

The following table provides a short prototype example of the base Random Forest parameters used in the current implementation:

Parameter	Current implementation
Number of trees	100
Split criterion	Gini impurity
Maximum tree depth	No predefined maximum
Minimum observations required to split a node	2
Minimum observations per terminal leaf	1

Parameter	Current implementation
Number of features considered at each split	Square root of the available features
Bootstrap sampling	Applied
Random seed	42

These parameters describe the current prototype and should not be interpreted as mandatory for every application of the protocol. Alternative parameter configurations or classification algorithms may be evaluated within the same training and comparison pipeline.

Feature Selection

The complete TF-IDF representation may contain a large number of features relative to the number of labeled statements. Feature selection may therefore be incorporated into model training to reduce noise, improve computational efficiency, and identify a more parsimonious representation.

The current implementation evaluated three general feature strategies:

Feature strategy	Description
Fixed feature selection	A predefined number of features is selected from the training data using an explicit statistical criterion.
Complete vocabulary	The classifier uses the complete TF-IDF feature space without a separate feature-selection stage.
Recursive feature elimination	The feature subset is selected iteratively according to model performance within cross-validation.

The implementation does not require researchers to evaluate these exact three strategies. However, when feature selection is used, it must be performed only with information from the training partition.

Feature selection must also be included inside the cross-validation process. Selecting features once from the complete training partition before cross-validation would allow information from the validation folds to influence the selected vocabulary. Instead, the selector should be fitted independently within each training fold and then applied to the corresponding validation fold.

Model Training

After the candidate model and feature-selection strategy have been specified, the resulting pipeline should be fitted using only observations assigned to the training partition. The pipeline includes the fitted TF-IDF vectorizer produced, any feature-selection procedure, and the classifier. The test partition must not be used to fit or modify any component of the pipeline.

When multiple configurations are evaluated, each configuration should be fitted independently using the same training observations and reproducible random seeds. Class-weight strategies and model parameters should be recorded as part of the model configuration. Feature selection and classifier fitting should subsequently be repeated within the internal cross-validation procedure described below.

Internal Cross-Validation

Model development should use stratified k-fold cross-validation within the training partition. Stratification preserves the relative distribution of regulative, constitutive, and non-institutional statements across the folds. The number of folds should be defined and reported for each implementation.

The same folds should be used when comparing model configurations whenever possible. This allows differences in performance to be attributed to the classifier, class-weight strategy, or feature-selection procedure rather than to different fold compositions.

For each cross-validation fold, the complete modeling pipeline should be reproduced:

1. Select the fold-specific training and validation observations.
2. Fit any feature-selection procedure using only the fold-specific training observations.
3. Apply the selected features to the fold-specific validation observations.
4. Fit the classifier using the fold-specific training observations.
5. Generate predictions for the fold-specific validation observations.
6. Calculate the evaluation metrics.
7. Store the fold assignments, selected features, predictions, and results.

When recursive feature elimination or another model-dependent feature-selection method is used, nested cross-validation is recommended. The inner cross-validation procedure selects the feature subset, while the outer cross-validation procedure estimates the performance of the complete feature-selection and classification pipeline.

Evaluation with the Test Partition

After internal model development, each candidate configuration may be fitted using the complete training partition and evaluated on the test partition. The fitted TF-IDF vectorizer and any feature-selection procedure created from the training partition must be applied to the test partition without refitting.

The following measures are used in the current implementation:

Indicator	Purpose in model comparison
Cross-validation macro-F1	Measures average internal performance across the five training folds while giving equal importance to the three classes.
Test macro-F1	Measures performance on the statements reserved in the test partition.
Combined macro-F1	Arithmetic mean of the cross-validation macro-F1 and test macro-F1.
Absolute performance gap	Absolute difference between cross-validation macro-F1 and test macro-F1; used as an indicator of performance stability.
Per-class precision	Measures how often predictions for each class are correct.
Per-class recall	Measures how many statements belonging to each class are identified.
Per-class F1	Summarizes precision and recall separately for each class.
Confusion matrix	Shows which classes are most frequently confused.
Accuracy and weighted F1	Retained as secondary descriptive measures.
Selected-feature count	Records model complexity and the degree of feature reduction.

Macro-F1 is the principal metric because each class receives equal importance despite differences in class frequency. Per-class results and the confusion matrix should always be inspected to ensure that a high aggregate result does not conceal poor performance for the constitutive or non-institutional classes.

The combined macro-F1 and absolute performance gap provide complementary information. The combined score summarizes performance across internal cross-validation and the test partition, while the gap indicates how consistently the model performs across both evaluation settings.

Model selection should also consider per-class results, confusion patterns, feature-set size, and methodological consistency with the intended application.

The complete comparison between candidate models, parameter configurations, class-weight strategies, feature-selection methods, and test results should be documented in a separate implementation or model-evaluation report. The protocol records the general comparison procedure but does not require the full experimental results to be reproduced in its main text.

Selected Model in the Current Implementation

For the current experimental implementation, the model-evaluation identifies an unweighted Random Forest using recursive feature elimination with cross-validation as the preferred model. The selection of this model is specific to the current dataset and implementation. It does not establish Random Forest or this feature subset as universally superior. The comparison procedure should be repeated when the labeled corpus changes substantially, when documents from other jurisdictions or document types are incorporated, or when alternative classifiers are introduced.

Model Outputs and Traceability

Each model-training run should create a model event that preserves the complete relationship between the labeled dataset, data split, vectorization event, feature-selection procedure, model configuration, and evaluation results.

Each training run is registered in `model_events`, which stores its upstream identifiers, candidate-model and feature-selection configuration, cross-validation design, evaluation summaries, selected configuration, technical validations, and status. External fitted models, predictions, selected-feature lists, reports, confusion matrices, fold results, and plots are registered in `model_artifacts` with their roles, paths, sizes, hashes, and timestamps.

The fitted model, feature selector, predictions, confusion matrices, selected-feature lists, evaluation reports, and other technical outputs should be stored as versioned model artifacts. Each artifact should retain its model identifier, artifact type, file path, integrity hash, and creation timestamp.

Once a model has been selected and its results reported, the corresponding training artifacts should not be overwritten. Any change to the dataset, vectorizer, features, parameters, classifier, class weights, or validation design should generate a new model event and new versioned artifacts.

Sub-task 3.5: Application to the Unlabeled Candidate Corpus

After a model has been selected, frozen, and versioned, it can be applied to the unlabeled candidate statements produced in Task 2. The application stage should preserve each original `statement_id` and `statement_text` and store the predicted class, available class scores or probabilities, confidence indicators, `model_id`, vectorization and dataset versions, prediction timestamp, and review status as a separate analytical output. Predictions must not overwrite human-validated `class_label` values. This final application step is specified here but has not yet been implemented in the current project.

Predictions should be treated as provisional coding proposals. Low-confidence or ambiguous cases can enter a human-review queue, and validated corrections can be incorporated into a new labeled-dataset version for later retraining. Each cycle should create new versioned splits, vectorizers, feature selectors, model events, artifacts, and prediction records rather than modifying earlier results.

REFERENCES

- Brandsma, G. J., Blom-Hansen, J., van Hal, F., & Moodley, K. (2025). A sentence-based approach to measuring EU regulatory activity. *Journal of European Public Policy*. Advance online publication. <https://doi.org/10.1080/13501763.2025.2596122>
- Bushouse, B. K., Virgüez-Ruiz, S., Chakraborti, M., & Frey, S. (2026). Advancing text analysis for nonprofit research: Using semantic role labeling to automate Institutional Grammar coding of nonprofit laws and policies. *Nonprofit Policy Forum*, 17(1–2), 323–341. <https://doi.org/10.1515/npf-2024-0048>
- Crawford, S. E. S., & Ostrom, E. (1995). A grammar of institutions. *American Political Science Review*, 89(3), 582–600. <https://doi.org/10.2307/2082975>
- Dunlop, C. A., Kamkhaji, J. C., & Radaelli, C. M. (2019). A sleeping giant awakes? The rise of the Institutional Grammar Tool (IGT) in policy research. *Journal of Chinese Governance*, 4(2), 163–180. <https://doi.org/10.1080/23812346.2019.1575502>
- Frantz, C. K., & Siddiki, S. (2021). Institutional Grammar 2.0: A specification for encoding and analyzing institutional design. *Public Administration*, 99(2), 222–247. <https://doi.org/10.1111/padm.12719>
- Galik, C. S., & Johnson, B. A. (2026). Governance of climate-induced relocation in the US: An exploration of institutional complexity. *Review of Policy Research*, 43(2), Article e70081. <https://doi.org/10.1111/ropr.70081>
- Guimarães, G. M. C., da Silva, F. X. B., Queiroz, A. L., Marcacini, R. M., Faleiros, T. P., Borges, V. R. P., & Garcia, L. P. F. (2024). DODFMiner: An automated tool for named entity recognition from official gazettes. *Neurocomputing*, 568, Article 127064. <https://doi.org/10.1016/j.neucom.2023.127064>
- Heikkilä, T., & Weible, C. M. (2018). A semiautomated approach to analyzing polycentricity. *Environmental Policy and Governance*, 28(4), 308–318. <https://doi.org/10.1002/eet.1817>
- Kim, M., & Collins, B. K. (2026). Regional transportation planning through intergovernmental collective action: Does planning conformity with federal regulations improve regional air quality? *Public Works Management & Policy*, 31(3), 324–342. <https://doi.org/10.1177/1087724X261421903>
- Pieper, L., Virgüez, S., Schlager, E., & Schweik, C. (2023). The use of the Institutional Grammar 1.0 for institutional analysis: A literature review. *International Journal of the Commons*, 17(1), 256–270. <https://doi.org/10.5334/ijc.1214>
- Rice, D., Siddiki, S., Frey, S., Kwon, J. H., & Sawyer, A. (2021). Machine coding of policy texts with the Institutional Grammar. *Public Administration*, 99(2), 248–262. <https://doi.org/10.1111/padm.12711>
- Saketos, V., Pantazi, D.-A., & Koubarakis, M. (2024). The large language model GreekLegalRoBERTa. In *Proceedings of the 13th Hellenic Conference on Artificial Intelligence* (Article 18). Association for Computing Machinery. <https://doi.org/10.1145/3688671.3688770>
- Siddiki, S., & Frantz, C. K. (2022). The institutional grammar in policy process research. *Policy Studies Journal*, 50(2), 299–314. <https://doi.org/10.1111/psj.12466>
- Turaga, V. S. P., Pahi, T., Tjoa, S., Siami-Namini, S., & Namin, A. S. (2025). CO2 (Co-Compliance Officer): An LLM-based ontology-driven methodology for generating knowledge graphs and AI compliance checking. *IEEE Access*, 13, 210576–210606. <https://doi.org/10.1109/ACCESS.2025.3639228>

- Vannoni, M. (2022). A political economy approach to the grammar of institutions: Theory and methods. *Policy Studies Journal*, 50(2), 453–471. <https://doi.org/10.1111/psj.12427>
- Wróblewska, A., Pielniński, B., Seweryn, K., Sysko-Romańczuk, S., Saputa, K., Wichrowska, A., & Schreiber, H. (2023). Automating the analysis of institutional design in international agreements. In A. Kő & E. Francesconi (Eds.), *Electronic government and the information systems perspective* (pp. 68–84). Springer. https://doi.org/10.1007/978-3-031-36024-4_5